This chapter introduces the fundamental concepts of program analysis, exploring how we can mathematically assign meaning to syntax and systematically reason about program behaviors.

---

## 1. Overview and Motivation

Program analysis is the systematic examination of a program to determine its properties. We apply it for various reasons:

- Finding bugs (historical fun fact: the first "bug" was an actual moth found in the Harvard Mark II in 1947).
- Optimizing performance and building compilers.
- Detecting security vulnerabilities and improving software maintainability.

To check if a program behaves as intended, we must verify if the code satisfies a specific semantic property. However, this is fundamentally constrained by **Rice's Theorem**: there exists no algorithm that can automatically, universally, and exactly determine if a program satisfies a non-trivial semantic property.

Because we cannot have a tool that is simultaneously automatic, universal, and exact, we must rely on different techniques that trade off these traits:

- **Testing:** Unsound (can accept incorrect programs), but complete (does not reject safe programs).
- **Machine-Assisted Proving (e.g., Coq, Isabelle):** Sound and quasi-complete, but requires manual effort to supply invariants.
- **Finite-State Model Checking:** Automatic, sound, and complete, but only with respect to finite models (can fail to terminate on infinite states).
- **Conservative Static Analysis:** Automatic and sound, but incomplete (relies on over-approximation).

---

## 2. Formal Semantics and State

Semantics assigns mathematical meaning to syntax. We distinguish between the syntactic representation of a program (how it is written, $c$) and its computed function ($[[c]]$).

- **Memory State:** A state $\sigma$ is a function mapping variables to integers: $\sigma : \mathbb{X} \to \mathbb{Z}$.
- **State Space:** The set of all possible states is denoted as $\Sigma \triangleq \{\sigma : \mathbb{X} \to \mathbb{Z}\}$.
- **State Update:** The notation $\sigma[n/\mathtt{x}]$ or $\sigma[\mathtt{x} \mapsto n]$ denotes a state identical to $\sigma$, except the variable $\mathtt{x}$ evaluates to $n$.

### 2.1 Forward Semantics

For deterministic programs, the denotational semantics models a program as a function from an input state to an output state: $[[c]] : \Sigma \to \Sigma_\perp$.

- The set $\Sigma_\perp = \Sigma \uplus \{\perp\}$ includes a special symbol $\perp$ to represent non-terminating executions.

### 2.2 Collecting Semantics

To consider all possible execution trajectories simultaneously (especially useful for non-deterministic programs or abstraction), we lift the semantics to operate on sets of states (the powerset $\wp(\Sigma)$).

- **Collecting Semantics:** $[[c]] : \wp(\Sigma) \to \wp(\Sigma)$.
- Defined as: $[[c]]P = \bigcup_{\sigma \in P} [[c]]\sigma$.

---

## 3. Approximations

Since exact analysis is impossible due to undecidability, we use approximations.

- **Over-approximations:** Expand the set of possible behaviors. If the over-approximated program doesn't reach a bad state, the exact program won't either.
  - *Pro:* Good for proving correctness (Sound).
  - *Con:* Can report bugs that don't exist (False Positives).
- **Under-approximations:** Restrict the set of possible behaviors. If the under-approximated program reaches a bad state, the exact program definitely will.
  - *Pro:* Good for bug-finding.
  - *Con:* Can miss actual bugs (False Negatives).

---

## 4. Denotational Semantics of IMP

We construct the meaning of a program compositionally.

### 4.1 Expressions

- **Arithmetic Expressions (Aexp):** $[[\cdot]] : \text{Aexp} \to \Sigma \to \mathbb{Z}$. Example: $[[\mathtt{a}_0 \text{ op } \mathtt{a}_1]]\sigma \triangleq [[\mathtt{a}_0]]\sigma \text{ op } [[\mathtt{a}_1]]\sigma$.
- **Boolean Expressions (Bexp):** In collecting semantics, a boolean expression filters a set of states, keeping only those where the condition holds: $[[\cdot]] : \text{Bexp} \to \wp(\Sigma) \to \wp(\Sigma)$. Example: $[[\mathtt{a}_0 \text{ cmp } \mathtt{a}_1]]P \triangleq \{\sigma \in P \mid [[\mathtt{a}_0]]\sigma \text{ cmp } [[\mathtt{a}_1]]\sigma\}$.

### 4.2 Commands and Control Flow

Command semantics $[[\cdot]] : \text{Com} \to \wp(\Sigma) \to \wp(\Sigma)$ are defined recursively:

- **Skip:** $[[\mathtt{skip}]]P \triangleq P$.
- **Assignment:** $[[\mathtt{x} := a]]P \triangleq \{\sigma[n/\mathtt{x}] \mid \sigma \in P, n = [[a]]\sigma\}$.
- **Sequence:** $[[c_1; c_2]]P \triangleq [[c_2]]([[c_1]]P)$.
- **Choice (Non-determinism):** $[[c_1 + c_2]]P \triangleq [[c_1]]P \cup [[c_2]]P$.

### 4.3 Loops and Fixpoints

Loops are handled using the **Kleene Star** operator, which applies a command zero or more times: $[[c^\star]]P \triangleq \bigcup_{k=0}^{\infty}[[c]]^k P$. To encode a `while` loop, we apply the loop body $c$ as long as condition $b$ holds, and finally filter by $\neg b$:

$$[[\mathtt{while\ b\ do\ c}]]P \triangleq [[\neg b]] \bigcup_{k=0}^{\infty} ([[c]] \circ [[b]])^k P$$

This represents finding the Least Fixed Point of the semantic function mapping states through iterations of the loop.

---

## 5. Correctness and Preconditions

We use semantic definitions to formally prove if a program starting in a set of input states $P$ will only result in valid output states $Q$.

- **Partial Correctness:** Denoted as $[[c]]P \subseteq Q$. This means that starting from $P$, the execution of $c$ either does not terminate, or terminates in a state satisfying $Q$.

To systematically calculate correctness, we compute preconditions:

### 5.1 Dijkstra's Weakest Liberal Precondition (wlp)

Calculates the largest set of input states from which execution *either* diverges *or* ends in $Q$:

$$wlp(c, Q) = \{\sigma \mid [[c]]\{\sigma\} \subseteq Q\}$$

Thus, $[[c]]P \subseteq Q$ if and only if $P \subseteq wlp(c, Q)$.

### 5.2 Backward Semantics and Weakest Possible Precondition (wpp)

Backward semantics reverses the relation: $[[c]]_{op} \triangleq \{(\delta, \sigma) \mid (\sigma, \delta) \in [[c]]\}$. Hoare's Weakest Possible Precondition calculates the largest set of input states that have *at least one successful computation* reaching $Q$:

$$wpp(c, Q) = \{\sigma \mid [[c]]\sigma \cap Q \neq \emptyset\}$$

Conveniently, this maps directly to the backward semantics: $wpp(c, Q) = [[c]]_{op}Q$.

# 02. Denotational Semantics

This chapter formalizes how to assign mathematical meaning to programs, mapping syntax to its computed function. It focuses on collecting semantics, which is the foundation for defining approximations in static analysis.

---

## 1. Preliminaries and State

Semantics assigns meaning to syntax. Given a program written in a specific syntax $c$, its meaning is denoted as $[[c]]$.

- **State ($\sigma$):** A concrete memory state is a function mapping variables to integers, $\sigma : \mathbb{X} \to \mathbb{Z}$.
- **State Space ($\Sigma$):** The set of all possible states is $\Sigma \triangleq \{\sigma : \mathbb{X} \to \mathbb{Z}\}$.
- **State Update:** The notation $\sigma[n/\mathtt{x}]$ or $\sigma[\mathtt{x} \mapsto n]$ indicates a state identical to $\sigma$, except the variable $\mathtt{x}$ has the value $n$.
- **Concrete Domain:** In program analysis, we often work with sets of states. The concrete domain is the powerset of $\Sigma$, denoted as $\wp(\Sigma) \triangleq \{P \mid P \subseteq \Sigma\}$.

---

## 2. Semantics of Expressions

Expressions are evaluated relative to a given state or set of states.

### 2.1 Arithmetic Expressions (`Aexp`)

The denotational semantics of an arithmetic expression evaluates to an integer given a specific state: $[[\cdot]] : \text{Aexp} \to \Sigma \to \mathbb{Z}$.

- $[[n]]\sigma \triangleq n$
- $[[\mathtt{x}]]\sigma \triangleq \sigma(\mathtt{x})$
- $[[\mathtt{a}_0 \text{ op } \mathtt{a}_1]]\sigma \triangleq [[\mathtt{a}_0]]\sigma \text{ op } [[\mathtt{a}_1]]\sigma$

### 2.2 Boolean Expressions (`Bexp`)

In collecting semantics, a boolean expression is treated as a filter over a set of states. It returns the subset of states that satisfy the condition: $[[\cdot]] : \text{Bexp} \to \wp(\Sigma) \to \wp(\Sigma)$.

- $[[\text{true}]]P \triangleq P$
- $[[\text{false}]]P \triangleq \emptyset$
- $[[\mathtt{a}_0 \text{ cmp } \mathtt{a}_1]]P \triangleq \{\sigma \in P \mid [[\mathtt{a}_0]]\sigma \text{ cmp } [[\mathtt{a}_1]]\sigma\}$
- $[[\text{not } b]]P \triangleq P \setminus ([[b]]P)$

*Note:* $[[b]]P \subseteq P$ always holds.

---

## 3. Regular Commands and Collecting Semantics

For deterministic programs, semantics can be defined as $[[c]] : \Sigma \to \Sigma_\perp$, where $\Sigma_\perp = \Sigma \uplus \{\perp\}$ handles non-termination. However, to handle non-determinism and lay the groundwork for abstraction, we use **Collecting Semantics**, which maps a set of input states to a set of output states: $[[c]] : \wp(\Sigma) \to \wp(\Sigma)$.

By definition, $[[c]]P = \bigcup_{\sigma \in P} [[c]]\{\sigma\}$.

### 3.1 Base Commands

- **Skip:** $[[\mathtt{skip}]]P \triangleq P$
- **Assignment:** $[[\mathtt{x} := a]]P \triangleq \{\sigma[n/\mathtt{x}] \mid \sigma \in P, n = [[a]]\sigma\}$
- **Guard/Filter:** $[[b?]]P \triangleq [[b]]P$
- **Sequence:** $[[c_1; c_2]]P \triangleq [[c_2]]([[c_1]]P)$
- **Non-deterministic Choice:** $[[c_1 + c_2]]P \triangleq [[c_1]]P \cup [[c_2]]P$

### 3.2 Loops and Kleene Star

To execute a command an arbitrary number of times, we use the Kleene star operator ($c^\star$).

- $[[c^\star]]P \triangleq \bigcup_{k=0}^{\infty} [[c]]^k P$

This is mathematically resolved using Kleene's fixpoint theorem. The semantics of $[[c^\star]]P$ represents the smallest set of states that contains the initial states $P$ and is closed under the execution of $c$: $[[c^\star]]P = P \cup [[c]]([[c^\star]]P)$

### 3.3 Encoding High-Level Control Flow

Standard control flow constructs are encoded using regular commands:

- **If-Then-Else:** $\mathtt{if\ b\ then\ c_1\ else\ c_2} \triangleq (b?; c_1) + (\neg b?; c_2)$
  - Semantics: $[[\mathtt{if\ b\ then\ c_1\ else\ c_2}]]P \triangleq [[c_1]]([[b]]P) \cup [[c_2]]([[\neg b]]P)$

- **While Loop:** `while b do c` $\triangleq (b?; c)^\star; \neg b?$
  - Semantics: $[[\texttt{while b do c}]]P \triangleq [[\neg b]] \left(\bigcup_{k=0}^{\infty}([[c]] \circ [[b]])^k P\right)$

---

## 4. Partial Correctness and Preconditions

We formalize correctness by checking if the reachable states conform to a specification.

**Partial Correctness:** Given a set of input states $P$ and admissible outputs $Q$, a program $c$ is partially correct if $[[c]]P \subseteq Q$. This means execution either diverges (does not terminate) or reaches a state in $Q$.

### 4.1 Dijkstra's Weakest Liberal Precondition (wlp)

The Weakest Liberal Precondition calculates the largest set of input states from which all terminating computations are guaranteed to end in $Q$.

$$wlp(c, Q) = \{\sigma \mid [[c]]\{\sigma\} \subseteq Q\}$$

*Theorem:* $[[c]]P \subseteq Q \iff P \subseteq wlp(c, Q)$.

### 4.2 Relational and Backward Semantics

Collecting semantics can be equivalently expressed as a relation $[[c]] \subseteq \Sigma \times \Sigma$, where $(\sigma, \delta) \in [[c]]$ means that starting from $\sigma$, state $\delta$ is reachable.

**Backward Semantics:** The opposite relation maps output states back to their possible input states.

$$[[c]]_{op} \triangleq \{(\delta, \sigma) \mid (\sigma, \delta) \in [[c]]\}$$

For example, $[[c_1; c_2]]_{op} \triangleq [[c_1]]_{op} \circ [[c_2]]_{op}$.

### 4.3 Hoare's Weakest Possible Precondition (wpp)

The Weakest Possible Precondition identifies the largest set of input states that have *at least one* successful computation ending in $Q$.

$$wpp(c, Q) = \{\sigma \mid [[c]]\{\sigma\} \cap Q \neq \emptyset\}$$

This directly corresponds to applying the backward semantics to the set of target states:

$wpp(c, Q) = [[c]]_{op} Q$.